THE FPU REPORT

1. Architecture of the Floating Point Unit (FPU)

The RISC201 Floating Point Unit (FPU) is a dedicated, standalone execution coprocessor engineered to natively support IEEE 754 Single-Precision arithmetic. Physically separated from the integer Arithmetic Logic Unit (ALU), the FPU ensures strict modularity and simplifies overarching pipeline control signals. By utilizing isolated, dedicated algorithmic hardware blocks—including a custom Exponent Subtractor, a Mantissa Kogge-Stone Adder, a 24-bit optimized Radix-4 Booth Multiplier, and a Fractional Non-Restoring Divider—the FPU maintains single-pass combinational execution without stalling the overarching pipeline. The unit internally manages all IEEE 754 sub-routines, including operand unpacking, implicit bit restoration, decimal alignment, and Leading Zero Detection (LZD) normalization.

2. FPU Instruction Datapaths and Hardware Routing

FADD & FSUB: Floating-Point Addition and Subtraction

Floating-point addition requires explicit decimal alignment before fractional magnitudes can be mathematically resolved. FSUB is executed by physically inverting the operand's sign wire and routing the data through the exact same FADD hardware sequence.

- **Unpacking & Alignment:** The hardware isolates the 8-bit exponents and 23-bit mantissas, appending the implicit 1 to generate a 24-bit operational fraction. A dedicated 8-bit Kogge-Stone subtractor calculates the absolute difference (ΔE) between the exponents. The 24-bit mantissa associated with the smaller exponent is routed into a Logarithmic Barrel Shifter, shifting right by ΔE positions to physically align the binary points.
- **Core Arithmetic:** The aligned 24-bit mantissas are routed into the FPU's dedicated Kogge-Stone Adder (or Subtractor, based on sign discrepancies).
- **Normalization:** A Leading Zero Detector (LZD) combinational array scans the resulting sum. The mantissa is routed back through the Barrel Shifter to reposition the first 1 to the Most Significant Bit (MSB), while the winning exponent is incrementally adjusted by the corresponding shift amount before the final 32-bit Word is packed.

FMUL: Floating-Point Multiplication

Multiplication bypasses the need for decimal alignment, allowing for a structurally direct hardware route.

- **Exponent Resolution:** The two biased 8-bit exponents are added together via an 8-bit Kogge-Stone adder. To correct the resultant double-bias ($+ 254$), a constant 127 is subtracted from the combined sum.

- **Core Arithmetic:** The two 24-bit mantissas are zero-padded to 32 bits and driven into the FPU's dedicated Radix-4 Booth Multiplier. Because the FPU is an isolated unit, the upper 8 bits are permanently hardwired to zero, allowing the Booth encoder to instantly bypass the higher stages and securely generate the 48-bit fractional product in a highly optimized single pass.
- **Normalization:** The hardware checks bit 47 of the resultant product. If the bit is high, the mantissa is shifted right by 1 position, and the calculated exponent is incremented to maintain scientific notation.

FDIV: Floating-Point Division (Fractional Optimization)

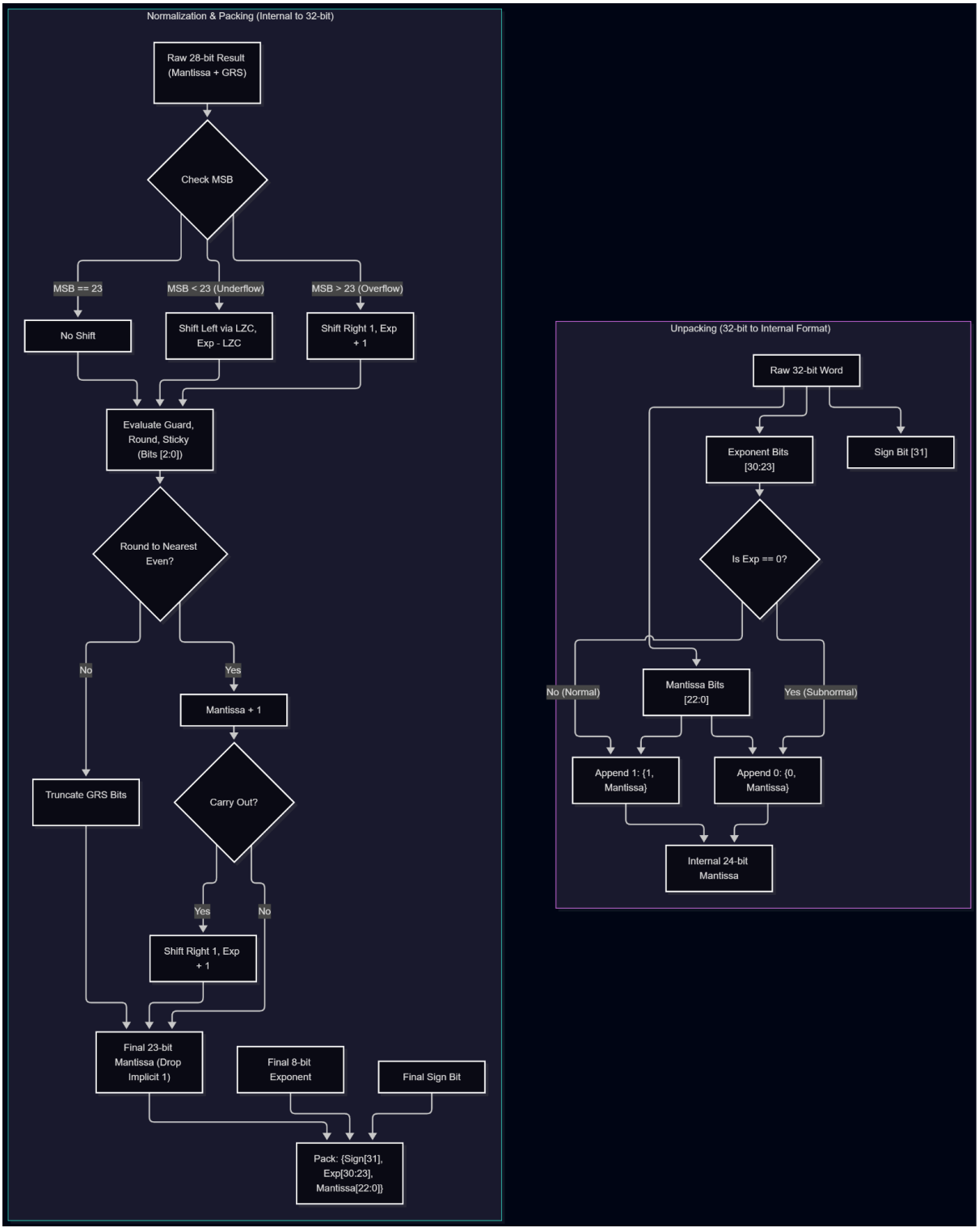
To natively calculate fractional quotients rather than integer quotients, the FPU utilizes a modified, hardwired version of the 32-stage Non-Restoring Divider.

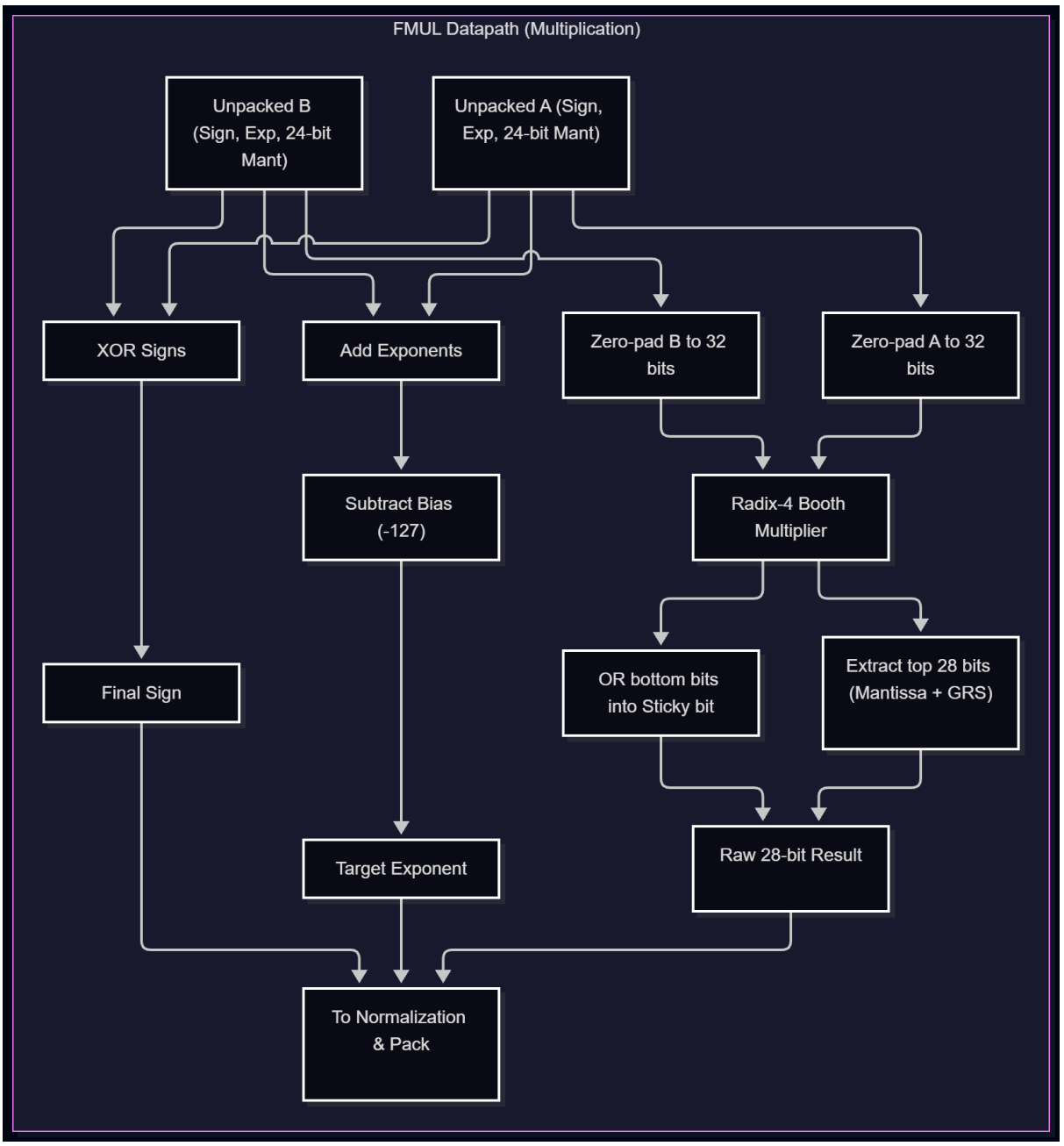
- **Exponent Resolution:** The divisor's exponent is physically subtracted from the dividend's exponent. The hardware subsequently adds the $+ 127$ bias back to the result to maintain standard IEEE 754 memory encoding.
- **Fractional Hardware Shift:** Unlike the integer ALU (which loads the dividend into the lower Q register), the FPU divider physically hardwires the incoming 24-bit dividend directly into the upper Accumulator register (A). This structural pre-shift effectively multiplies the dividend by 2^{32} prior to execution, guaranteeing that the divider cascade produces a high-precision fractional quotient rather than a truncated integer.
- **Normalization:** The hardware inspects the MSB of the generated quotient, applying a left shift and exponent decrement if the leading bit evaluates to 0 prior to IEEE 754 repacking.

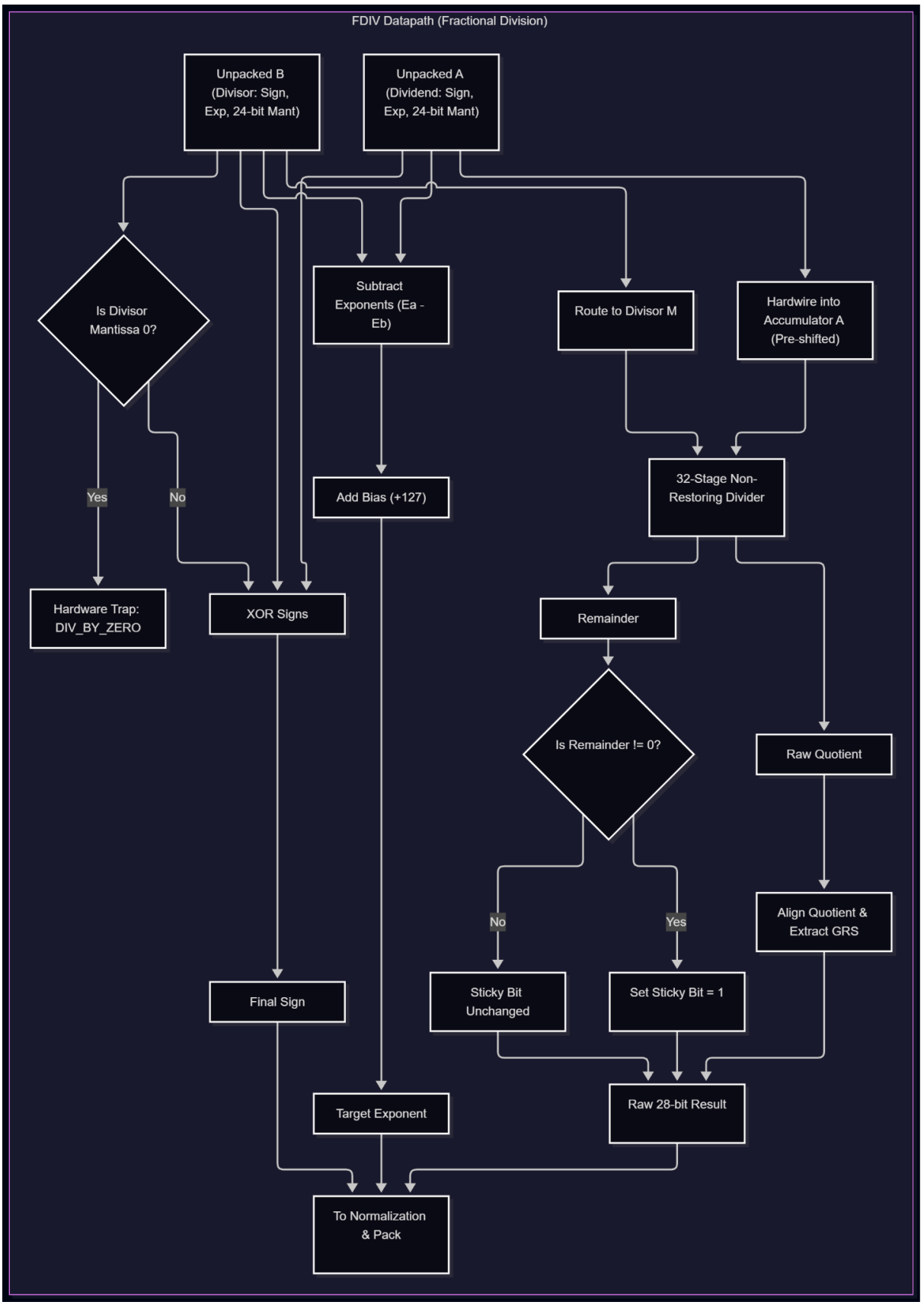
FCMP: Floating-Point Compare

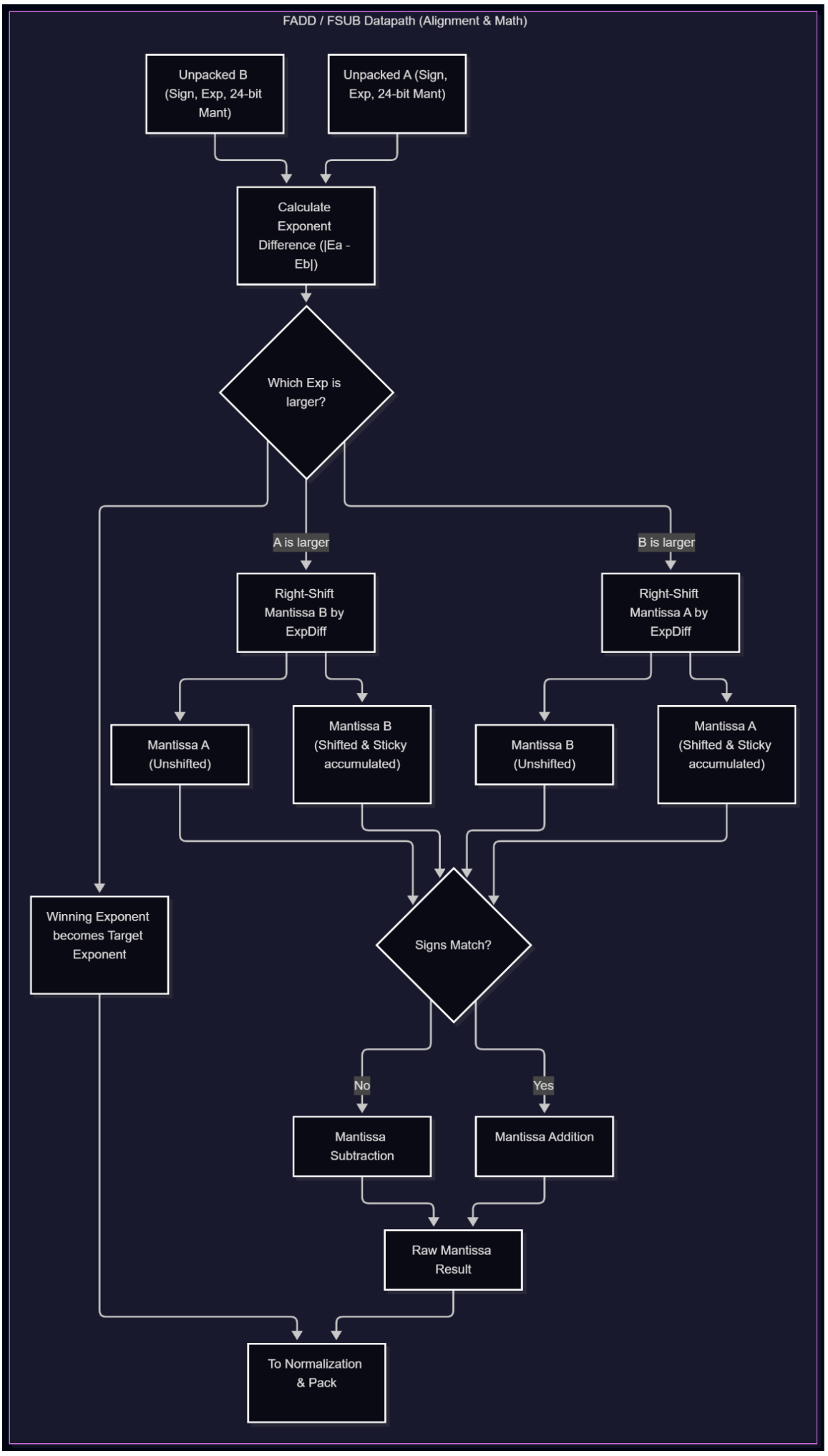
The compare instruction evaluates conditions and updates the overarching processor pipeline flags (Z, N, NaN) without committing mathematical results back to the Register File.

- **Exception Intercept:** The hardware instantly inspects both unpacked operands. If either exponent bus reads 11111111 with a non-zero mantissa, the unit immediately triggers the hardware NaN (Not a Number) status flag.
- **Phantom Subtraction:** Valid operands pass through the standard FSUB exponent alignment and Kogge-Stone subtraction datapath.
- **Flag Generation:** The output data line is physically grounded to zero to prevent register writes. The Negative (N) flag is tapped directly from the FPU subtractor's sign bit, and the resulting mantissa is driven through a massive NOR gate array to evaluate and set the Zero (Z) flag.


Normalization & Packing (Internal to 32-bit)
Raw 28-bit Result (Mantissa + GRS)
Check MSB
MSB == 23
MSB < 23 (Underflow)
MSB > 23 (Overflow)
No Shift
Shift Left via LZC, Exp - LZC
Shift Right 1, Exp + 1
Evaluate Guard, Round, Sticky (Bits [2:0])
Round to Nearest Even?
No
Yes
Mantissa + 1
Truncate GRS Bits
Carry Out?
Yes
No
Shift Right 1, Exp + 1
Final 23-bit Mantissa (Drop Implicit 1)
Final 8-bit Exponent
Final Sign Bit
Pack: {Sign[31], Exp[30:23], Mantissa[22:0]}
Unpacking (32-bit to Internal Format)
Raw 32-bit Word
Exponent Bits [30:23]
Sign Bit [31]
Is Exp == 0?
No (Normal)
Mantissa Bits [22:0]
Yes (Subnormal)
Append 1: {1, Mantissa}
Append 0: {0, Mantissa}
Internal 24-bit Mantissa


FMUL Datapath (Multiplication)
Unpacked B (Sign, Exp, 24-bit Mant)
Unpacked A (Sign, Exp, 24-bit Mant)
XOR Signs
Add Exponents
Zero-pad B to 32 bits
Zero-pad A to 32 bits
Subtract Bias (-127)
Radix-4 Booth Multiplier
Final Sign
OR bottom bits into Sticky bit
Extract top 28 bits (Mantissa + GRS)
Target Exponent
Raw 28-bit Result
To Normalization & Pack


FDIV Datapath (Fractional Division)
Unpacked B (Divisor: Sign, Exp, 24-bit Mant)
Unpacked A (Dividend: Sign, Exp, 24-bit Mant)
Is Divisor Mantissa 0?
Subtract Exponents (Ea - Eb)
Route to Divisor M
Hardwire into Accumulator A (Pre-shifted)
Yes
No
Add Bias (+127)
32-Stage Non-Restoring Divider
Hardware Trap: DIV_BY_ZERO
XOR Signs
Remainder
Is Remainder != 0?
Raw Quotient
No
Yes
Align Quotient & Extract GRS
Final Sign
Sticky Bit Unchanged
Set Sticky Bit = 1
Target Exponent
Raw 28-bit Result
To Normalization & Pack


FADD / FSUB Datapath (Alignment & Math)
Unpacked B (Sign, Exp, 24-bit Mant)
Unpacked A (Sign, Exp, 24-bit Mant)
Calculate Exponent Difference (|Ea - Eb|)
Which Exp is larger?
A is larger
B is larger
Right-Shift Mantissa B by ExpDiff
Right-Shift Mantissa A by ExpDiff
Mantissa A (Unshifted)
Mantissa B (Shifted & Sticky accumulated)
Mantissa B (Unshifted)
Mantissa A (Shifted & Sticky accumulated)
Winning Exponent becomes Target Exponent
Signs Match?
No
Yes
Mantissa Subtraction
Mantissa Addition
Raw Mantissa Result
To Normalization & Pack